

React.js Interview Preparation Guide

1. Hooks (prepare in this order)

useState

- Adds local state to function components. Returns `[state, setState]` .
- State updates are **asynchronous** and **batched** (React 18 batches even in timeouts/promises).
- Functional updates: `setCount(prev => prev + 1)` — use when new state depends on old state.

```
1  const [count, setCount] = useState(0);
2  setCount(prev => prev + 1);
```

Common questions: Why use functional updater form? Why doesn't state update immediately after `setState` ? Difference between `useState` and `this.state` in class components?

useEffect

- Runs side effects (data fetching, subscriptions, DOM manipulation) after render.
- Dependency array controls when it re-runs: `[]` = once on mount, `[dep]` = on dep change, no array = every render.
- Cleanup function (`return () => {...}`) runs before re-run and on unmount — used to cancel subscriptions/timers.

```
1  useEffect(() => {
2    const id = setInterval(tick, 1000);
3    return () => clearInterval(id);
4  }, []);
```

Common questions: Difference between `useEffect` and `useLayoutEffect` ? What happens if you omit the dependency array? Stale closure problem — how to avoid it?

useContext

- Consumes a Context value without wrapping in `<Context.Consumer>` .
- Avoids prop drilling for global-ish data (theme, auth, locale).

```
1  const theme = useContext(ThemeContext);
```

Common questions: When does a component re-render on context change? How to avoid unnecessary re-

renders with Context (splitting contexts, memoization)?

useReducer

- Alternative to `useState` for complex state logic (multiple sub-values, next state depends on previous).
- Pattern: `(state, action) => newState`.

```
1 function reducer(state, action) {  
2   switch (action.type) {  
3     case 'increment': return { count: state.count + 1 };  
4     default: throw new Error();  
5   }  
6 }  
7 const [state, dispatch] = useReducer(reducer, { count: 0 });
```

Common questions: `useState` vs `useReducer` — when to pick which? How does `useReducer` relate to Redux's pattern?

useMemo

- Memoizes a **computed value** so it's only recalculated when dependencies change. Optimizes expensive calculations.

```
1 const expensiveValue = useMemo(() => computeHeavy(a, b), [a, b]);
```

Common questions: When does over-using `useMemo` hurt performance? Difference between `useMemo` and `useCallback`?

useCallback

- Memoizes a **function reference** so it doesn't get recreated on every render — important when passing callbacks to memoized child components (`React.memo`).

```
1 const handleClick = useCallback(() => doSomething(id), [id]);
```

Common questions: Why does a new function reference cause child re-renders? How does `useCallback` pair with `React.memo`?

useRef

- Persists a mutable value across renders **without** causing re-render.
- Common uses: DOM element access, storing previous values, timers/intervals, avoiding stale closures.

```
1  const inputRef = useRef(null);
2  useEffect(() => { inputRef.current.focus(); }, []);
```

Common questions: Difference between useRef and useState? Can useRef trigger a re-render? How to access a DOM node?

2. Higher Order Components (HOC)

- ```
const Enhanced = withX(Component);
```
- **When?** Reusing component logic across multiple components (auth checks, logging, data fetching, theming) before hooks existed — still used in some libraries ( `connect()` in Redux, `withRouter` ).
- **Why?** Code reuse, separation of concerns, cross-cutting logic without duplicating code.
- **How?**

```
1 function withAuth(WrappedComponent) {
2 return function AuthenticatedComponent(props) {
3 const isAuth = useAuth();
4 if (!isAuth) return <Redirect to="/login" />;
5 return <WrappedComponent {...props} />;
6 };
7 }
8 const ProtectedDashboard = withAuth(Dashboard);
```

**Talking point:** Compare HOC vs Custom Hooks vs Render Props — modern React favors custom hooks for logic reuse since they avoid "wrapper hell" and prop-name collisions.

---

## 3. Lifecycle Methods (Class Components)

---

### Mounting

```
constructor()
static getDerivedStateFromProps()
render()
componentDidMount() — API calls, subscriptions, DOM refs.
```

### Updating

```
static getDerivedStateFromProps()
shouldComponentUpdate() — performance optimization, return false to skip render.
```

`render()`

`getSnapshotBeforeUpdate()`

`componentDidUpdate()` — react to prop/state changes, compare with previous props.

## Unmounting

- `componentWillUnmount()` — cleanup (timers, subscriptions, listeners).

Hook equivalents (frequently asked):

| Class                              | Hook                                           |
|------------------------------------|------------------------------------------------|
| <code>componentDidMount</code>     | <code>useEffect(() =&gt; {...}, [])</code>     |
| <code>componentDidUpdate</code>    | <code>useEffect(() =&gt; {...}, [dep])</code>  |
| <code>componentWillUnmount</code>  | cleanup function in <code>useEffect</code>     |
| <code>shouldComponentUpdate</code> | <code>React.memo</code> / <code>useMemo</code> |

---

## 4. State Management (all about data)

### State vs Props

- **Props:** read-only, passed from parent to child.
- **State:** local, mutable (via setter), owned by the component.

### Props Drilling

- Passing props through many intermediate components that don't need them just to reach a deeply nested child.
- Problem: hard to maintain, unnecessary re-renders, tight coupling.

### Context

- Solves prop drilling for "global" data by letting any descendant subscribe directly.
- Trade-off: overusing Context can cause broad re-renders; not a full state-management replacement for complex/frequently-changing state (use Redux/Zustand for that).

---

## 5. Redux or Zustand

### How Redux works

- Single **store** holds global state (tree).
- Components **dispatch actions** (plain objects { type, payload } ).
- **Reducers** (pure functions) compute new state from action + previous state.
- Store notifies subscribed components → re-render.
- Flow: View → dispatch(action) → reducer → new state → store → View

## Why Redux?

- Predictable state container, single source of truth, great devtools (time-travel debugging), good for large apps with complex shared state across many components.

## When to use it?

- Multiple unrelated components need the same data, complex state transitions, need for middleware (logging, async side-effects via thunk/saga).
- For simpler needs, **Zustand** offers a lighter API (no boilerplate, no Provider needed, hook-based store) — increasingly preferred for medium apps.

## Redux Toolkit (RTK)

- Official, opinionated way to write Redux: `configureStore`, `createSlice` (combines actions + reducers), built-in `immer` for "mutating" syntax that's actually immutable under the hood, `createAsyncThunk` for async logic, RTK Query for data fetching/caching.

```

1 const counterSlice = createSlice({
2 name: 'counter',
3 initialState: { value: 0 },
4 reducers: {
5 increment: (state) => { state.value += 1; } // immer handles immutability
6 }
7 });

```

**Talking point:** RTK removes most Redux boilerplate (action creators, switch-case reducers) and is now the recommended standard.

## 6. Custom Hooks

### When to use?

- When the same stateful logic (data fetching, form handling, window size, debounce, subscriptions) is repeated across multiple components.

## Code

```

1 function useFetch(url) {
2 const [data, setData] = useState(null);
3 const [loading, setLoading] = useState(true);
4 const [error, setError] = useState(null);
5
6 useEffect(() => {
7 let cancelled = false;
8 setLoading(true);
9 fetch(url)
10 .then(res => res.json())
11 .then(json => { if (!cancelled) { setData(json); setLoading(false); } })
12 .catch(err => { if (!cancelled) { setError(err); setLoading(false); } });
13 return () => { cancelled = true; };
14 }, [url]);
15
16 return { data, loading, error };
17 }
18
19 // Usage
20 const { data, loading, error } = useFetch('/api/users');

```

## Why?

- Makes code **clean, maintainable, readable, reusable** — extracts logic out of components so components stay focused on rendering (UI), while hooks own the "how".
- **Interview hack:** Mention that you write custom hooks specifically to keep components testable in isolation (mock the hook return value in tests).

## 7. Lazy Loading (very important, highly asked)

### Code Splitting

- Breaking a large bundle into smaller chunks loaded on demand instead of one giant bundle upfront — improves initial load time.

### Chunking

- Bundlers (Webpack/Vite) create separate JS "chunks" per dynamic `import()` call; each route/feature can be its own chunk.

### Suspense

- `React.lazy()` + `<Suspense fallback={...}>` lets you lazily load a component and show a fallback UI while it loads.

```

1 const Dashboard = React.lazy(() => import('./Dashboard'));
2
3 function App() {
4 return (
5 <Suspense fallback={<Spinner />}>
6 <Dashboard />
7 </Suspense>
8);
9 }

```

**Common questions:** Where do you apply lazy loading in a real app (route-level, heavy modals, charts)? How does Suspense interact with data fetching (React 18 Suspense for data)? What happens if the lazy import fails (error boundaries)?

## 8. Virtual DOM

- A lightweight, in-memory JS representation of the real DOM. React builds a new virtual DOM tree on every state/prop change and compares it with the previous one — this comparison is the **diffing** process.

### Diff Algorithm

- Compares old vs new tree **level by level** ( $O(n)$  instead of  $O(n^3)$  full tree comparison).
- Elements of different types → old subtree torn down, new one built.
- Same type → attributes updated in place; children diffed using **keys** (hence "always use stable keys, never index" as a common gotcha).

### Reconciliation Algorithm

- The process of applying the minimal set of DOM operations needed based on the diff, updating the real DOM efficiently instead of re-rendering everything.

### React Fiber

- React's reconciliation engine (since React 16) — reworks the reconciliation process into an incremental, **interruptible** unit-of-work model.
- Enables **prioritization** (urgent updates like typing vs. non-urgent like a data fetch), time-slicing, and features like Concurrent Mode / Suspense.

### How does render work?

State/prop change triggers a re-render (render phase — pure, no side effects, can be paused/resumed by Fiber).

React builds new virtual DOM tree, diffs against previous.

Commit phase — actual DOM mutations applied, refs updated, effects ( `useLayoutEffect` then

`useEffect` ) run.

**Common questions:** Does a re-render always mean a DOM update? (No — only if the diff finds changes.)  
Why do keys matter in lists?

---

## 9. SSR vs CSR (important)

---

### What?

- **CSR (Client-Side Rendering):** Browser downloads a mostly-empty HTML + JS bundle; JS renders the UI in the browser (classic Create React App style SPA).
- **SSR (Server-Side Rendering):** Server renders full HTML for each request and sends it to the browser; React then "hydrates" it (attaches event listeners) on the client (Next.js is the common tool).

### Difference

|               | CSR                                       | SSR                             |
|---------------|-------------------------------------------|---------------------------------|
| Initial load  | Slower first paint (blank until JS loads) | Faster first paint (HTML ready) |
| SEO           | Weaker (crawlers may not execute JS)      | Strong (full HTML available)    |
| Server load   | Low                                       | Higher (rendering per request)  |
| Interactivity | Immediate once JS loads                   | Needs hydration first           |

### SEO and Performance (SSR)

- SSR improves SEO since crawlers get fully-rendered HTML immediately, and improves perceived performance metrics like First Contentful Paint (FCP).
- Related concepts often asked together: **SSG** (Static Site Generation — pre-rendered at build time) and **ISR** (Incremental Static Regeneration).

---

## 10. Routing (with Role-Based Access Control - RBAC)

---

### react-router basics

- `<BrowserRouter>` , `<Routes>` , `<Route path="..." element={...} />` , `useNavigate()` , `useParams()` , `useSearchParams()` .

### Protected Routes

```

1 function ProtectedRoute({ children, allowedRoles }) {
2 const { user } = useAuth();
3 if (!user) return <Navigate to="/login" replace />;
4 if (allowedRoles && !allowedRoles.includes(user.role)) {
5 return <Navigate to="/unauthorized" replace />;
6 }
7 return children;
8 }
9
10 // Usage
11 <Route path="/admin" element={
12 <ProtectedRoute allowedRoles={['admin']}>
13 <AdminPanel />
14 </ProtectedRoute>
15 } />

```

## How do you handle routes?

- Centralize route config (array of { path, element, roles } ), map it to `<Route>` elements, use nested routes + `<Outlet />` for layouts.

## Query Params

```

1 const [searchParams, setSearchParams] = useSearchParams();
2 const page = searchParams.get('page');

```

## Dynamic Routing

```

1 <Route path="/users/:userId" element={<UserProfile />} />
2 // inside component:
3 const { userId } = useParams();

```

**Common questions:** How would you implement RBAC across nested routes? How do you handle 404s and unauthorized redirects? Lazy-load route components with `React.lazy` + `Suspense` .

# 11. Testing

## React Testing Library (RTL)

- Philosophy: test components the way a **user** interacts with them (by role/text/label), not implementation details.

```
1 import { render, screen, fireEvent } from '@testing-library/react';
2
3 test('increments counter on click', () => {
4 render(<Counter />);
5 fireEvent.click(screen.getByRole('button', { name: /increment/i }));
6 expect(screen.getByText('1')).toBeInTheDocument();
7 });
```

## Unit Testing

- Test pure functions, reducers, custom hooks ( `renderHook` from RTL), and isolated components with mocked dependencies (e.g., mock `fetch` /API layer).

**Interview hack:** When discussing any code you write, explicitly say: *"I structured this as a small pure function / custom hook so it's easy to unit test in isolation, and the component itself stays testable through RTL by querying accessible roles."* This signals testable, production-grade thinking.

---

## 12. Async Tasks

### API Calls

- `fetch` / `axios` inside `useEffect` , or via a data-fetching library (React Query / RTK Query) for caching, retries, and stale-while-revalidate behavior.

### useEffect in depth

- Runs after paint (commit phase). Cleanup avoids race conditions (e.g., component unmounts mid-fetch) and memory leaks.
- Common bug: **stale closures** — values captured at render time inside the effect don't reflect later updates unless included in the dependency array.

### Events

- Synthetic events in React normalize cross-browser behavior; event pooling was removed in React 17+.

### Promises

- `.then/.catch` vs `async/await` inside effects (can't make the effect callback itself `async` , so define an inner async function and call it).

```
1 useEffect(() => {
2 async function load() {
3 const res = await fetch(url);
4 const json = await res.json();
5 setData(json);
6 }
7 load();
8 }, [url]);
```

## setTimeout

- Common in debouncing, delayed UI states; always clear timers in the cleanup function to avoid setting state on an unmounted component.

---

## 13. Coding Practices — Reusability, Readability, Modularity, Testability

---

- **Reusability:** extract shared logic into custom hooks/utility functions; keep components generic via props/composition.
- **Readability:** clear naming, small functions/components, consistent formatting, avoid deeply nested ternaries/JSX.
- **Modularity:** separate concerns — UI components, business logic (hooks/services), API layer, types — feature-based folder structure.
- **Testability:** pure functions where possible, dependency injection (pass API client as prop/hook rather than hardcoding), avoid side effects buried deep in components.

---

## 14. Performance

---

- **Lazy loading:** route/component-level code splitting (see Section 7).
- **Asset optimization:** minify JS/CSS, tree-shaking unused code, compress images (WebP/AVIF), use responsive `srcset`, remove unused CSS.
- **Writing optimized code:** avoid unnecessary re-renders ( `React.memo`, `useMemo`, `useCallback` ), avoid creating new objects/functions inline in JSX props, virtualize long lists ( `react-window` ).
- **Bundler:** Webpack/Vite — code splitting, tree-shaking, minification, caching via content hashes in filenames.
- **CDN / Server level:** serve static assets via CDN for lower latency, enable gzip/Brotli compression, HTTP caching headers, HTTP/2.
- **Rendering of components:** avoid prop-drilling causing broad re-renders, split contexts, use keys correctly in lists, consider concurrent features ( `useTransition`, `useDeferredValue` ) for non-urgent updates.

---

## 15. Styling

---

| Approach          | Notes                                                                               |
|-------------------|-------------------------------------------------------------------------------------|
| Tailwind CSS      | Utility-first, no context switching, purges unused classes for small bundle         |
| StyleX            | Meta's compile-time CSS-in-JS, atomic styles, strong performance guarantees         |
| Bootstrap         | Component + grid based, quick to prototype, less customizable out of the box        |
| Material UI (MUI) | Full component library implementing Google's Material Design, theming API           |
| Ant Design        | Enterprise-focused component library, strong for admin dashboards/forms             |
| CSS/SCSS          | Full control, SCSS adds variables/nesting/mixins; needs BEM or CSS Modules to avoid |

**Common question:** How do you avoid CSS class name collisions at scale? (CSS Modules, CSS-in-JS scoping, BEM convention, or utility-first approaches like Tailwind.)

---

## 16. Accessibility (a11y)

---

- Semantic HTML ( `<button>` , `<nav>` , `<label>` ) over generic `<div>` s with click handlers.
  - ARIA attributes ( `aria-label` , `aria-live` , `role` ) when semantic HTML isn't enough.
  - Keyboard navigability (focus management, `tabIndex` , visible focus states).
  - Sufficient color contrast, alt text for images.
  - Testing tools: `axe-core` , Lighthouse accessibility audit, `eslint-plugin-jsx-a11y` .
- 

## 17. Security

---

- **XSS:** avoid `dangerouslySetInnerHTML` unless sanitized (e.g., with `DOMPurify`); React escapes values by default in JSX.
- **CSRF:** use anti-CSRF tokens for state-changing requests, `SameSite` cookies.
- **Dependency vulnerabilities:** regularly audit ( `npm audit` ), keep packages updated.
- **Sensitive data:** never store tokens/secrets in `localStorage` if avoidable (XSS can read it); prefer `httpOnly` secure cookies.
- **Environment variables:** never expose secret keys in client-side bundles (anything in a React build is publicly visible).
- **Content Security Policy (CSP):** restrict allowed script/style sources.

---

## Quick Final Tips

---

- Be ready to **write live code** for: custom hook, debounce hook, protected route, lazy-loaded component with Suspense, a `useReducer` counter/todo app.
- Always narrate **why** you're making a choice (performance, readability, testability) — interviewers weight reasoning as much as syntax.
- Have one real project story ready where you applied: code splitting, Redux/Zustand, or a custom hook — concrete impact (e.g., reduced bundle size, fixed re-render bug) lands well.